📊 Statystyka opisowa i eksploracyjna analiza danych

Czyli jak zaprzyjaźnić się z danymi, zanim zaczniesz je "trenować" 🤖

Python w Zastosowaniach Big Data | Wykład 1

Prowadzący: Pavlo Melnyk

📋 Plan wykładu

1. Dlaczego statystyka jest fundamentem Big Data i Data Science
2. Populacja, próba i typy zmiennych
3. Statystyka opisowa — miary tendencji centralnej i rozproszenia
4. Rozkłady prawdopodobieństwa
5. Eksploracyjna analiza danych (EDA) w pandas
6. Wizualizacja — histogramy, boxploty, heatmapy
7. Częste pułapki i jak ich unikać
8. Wnioskowanie statystyczne w pigułce — przedział ufności i p-wartość

> 🎯 **Po tym wykładzie:** Spojrzysz na dowolny zbiór danych i w 15 minut powiesz o nim coś sensownego — zanim jeszcze pomyślisz o Machine Learningu!

💡 Dlaczego to ważne?

Wyobraź sobie, że dostajesz plik z **1 500 000 wierszy** i zero opisu kolumn.

Co robisz? Od razu odpalasz `model.fit()`? 😱

NIE. Najpierw robisz to, co robi każdy dobry data scientist — **eksplorujesz dane**.

Bez EDA	Z EDA
Model "dziwnie" się zachowuje	Wiesz, że kolumna ma 40% braków
Wyniki nie mają sensu	Widzisz outliery zanim zepsują średnią
Godziny debugowania	Minuty na `df.describe()`

> Statystyka to **język, którym mówią dane**. Bez niego jesteś turystą bez rozmówek. 🗺️

🌍 Statystyka a Big Data — spojler na resztę semestru

Za kilka wykładów będziemy liczyć te same rzeczy na **miliardach rekordów** w Apache Spark.

Dobra wiadomość: **koncepcje się nie zmieniają** — zmienia się tylko skala!

```
pandas:  df['czas'].mean()          # miliony wierszy, jedna maszyna
spark:   df.select(avg("czas"))     # miliardy wierszy, klaster maszyn
```

> 🎯 Jeśli dziś zrozumiesz średnią, wariancję i rozkład normalny — za 5 wykładów zrozumiesz to samo na Sparku. Tylko szybciej. ⚡

🧮 Populacja vs próba

- **Populacja** — wszystkie możliwe obserwacje (np. wszyscy biegacze na świecie)
- **Próba (sample)** — podzbiór, na którym faktycznie pracujemy (np. 1200 zawodników jednego maratonu)

```
import numpy as np

populacja = np.random.normal(loc=250, scale=40, size=10_000_000)  # cała populacja
proba = np.random.choice(populacja, size=1200, replace=False)     # nasza próba
```

> 💡 Prawie zawsze pracujemy na **próbie** i staramy się wnioskować o **populacji**. To jest właśnie sedno statystyki!

🏷️ Typy zmiennych

Typ	Opis	Przykład
Numeryczna ciągła	Dowolna wartość w zakresie	czas ukończenia (min)
Numeryczna dyskretna	Wartości całkowite, policzalne	liczba maratonów w karierze
Kategoryczna nominalna	Kategorie bez porządku	płeć, miasto startu
Kategoryczna porządkowa	Kategorie z naturalnym porządkiem	kategoria wiekowa (18-29, 30-39...)

```
df.dtypes           # jaki typ ma każda kolumna?
df['plec'] = df['plec'].astype('category')  # jawne wskazanie kategorii
```

> 🤔 **Pytanie:** Czy "kategoria wiekowa" to zmienna numeryczna czy kategoryczna? (Podchwytliwe!)

📊 Statystyka opisowa — o co chodzi?

Statystyka opisowa podsumowuje dane, które MASZ — bez wyciągania wniosków poza próbę.

Trzy pytania, na które odpowiada:

1. Gdzie jest **środek** danych? (tendencja centralna)
2. Jak bardzo dane są **rozrzucone**? (rozproszenie)
3. Jaki jest **kształt** rozkładu? (symetria, ogony, outliery)

> To jest Twój pierwszy przystanek przy KAŻDYM nowym zbiorze danych. Zawsze. 🚦

🎯 Średnia (mean)

```
import numpy as np

czasy = np.array([210, 245, 198, 320, 255])
srednia = czasy.mean()
print(f"Średni czas: {srednia:.1f} min")  # 245.6 min
```

Średnia = suma / liczba obserwacji.

> ⚠️ **Problem:** Średnia jest **bardzo wrażliwa** na wartości odstające (outliery)!

```
czasy_z_outlierem = np.array([210, 245, 198, 320, 900])  # ktoś szedł, nie biegł 😅
print(czasy_z_outlierem.mean())  # 374.6 — kompletnie nierealistyczne!
```

🎯 Mediana — odporna na outliery

Mediana = środkowa wartość po posortowaniu danych.

```
mediana = np.median(czasy_z_outlierem)
print(f"Mediana: {mediana}")  # 245 — dużo bardziej sensowne!
```

Miara	Wrażliwość na outliery	Kiedy używać
Średnia	Wysoka	Dane symetryczne, bez outlierów
Mediana	Niska	Dane skośne (np. dochody, czasy biegu z DNF)

> 💡 **Zasada kciuka:** Zawsze porównuj średnią i medianę. Duża różnica = masz skośność albo outliery!

🎯 Moda (mode)

Moda = najczęściej występująca wartość. Jedyna miara centralna działająca też dla danych kategorycznych!

```
import pandas as pd

df['kategoria_wiekowa'].mode()[0]  # np. "30-39" — najliczniejsza grupa
```

```
# Przykład: najpopularniejszy numer buta na starcie maratonu
rozmiary = pd.Series([42, 43, 42, 41, 42, 44, 43, 42])
print(rozmiary.mode())  # 0    42
```

> 🎉 Moda to jedyna z "wielkiej trójki", która ma sens dla zmiennych typu "kolor koszulki" czy "miasto".

📏 Wariancja i odchylenie standardowe

Wariancja (σ^2) mierzy, jak bardzo dane odchylają się od średniej:

$$\sigma^2 = \frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^2$$

Odchylenie standardowe (σ) to pierwiastek z wariancji — wraca do jednostek oryginalnych danych!

```
czasy = np.array([210, 245, 198, 320, 255])
print(f"Wariancja: {czasy.var():.1f}")       # w minutach²  — trudne do interpretacji
print(f"Odch. std: {czasy.std():.1f} min")   # w minutach — czytelne!
```

> 💡 Wariancja jest w "kwadratowych" jednostkach — dlatego prawie zawsze patrzymy na **odchylenie standardowe**.

📐 Kwantyle i percentyle

Kwantyle dzielą posortowane dane na równe części.

```
czasy = np.random.normal(250, 40, 1200)

q1, q2, q3 = np.quantile(czasy, [0.25, 0.5, 0.75])
print(f"Q1={q1:.0f}, mediana={q2:.0f}, Q3={q3:.0f}")
```

Kwantyl	Nazwa	Znaczenie
Q1 (25%)	Pierwszy kwartyl	25% wyników jest poniżej
Q2 (50%)	Mediana	Środek danych
Q3 (75%)	Trzeci kwartyl	75% wyników jest poniżej

IQR (Interquartile Range) = Q3 − Q1 → podstawa do wykrywania outlierów!

📦 Boxplot — kwantyle w jednym obrazku

```
import matplotlib.pyplot as plt

plt.boxplot(czasy, vert=False)
plt.xlabel("Czas ukończenia (min)")
plt.title("Rozkład czasów maratończyków")
plt.show()
```

```
       Q1        mediana      Q3
        |            |          |
  *--[=========|==========]-----*
  ↑                            ↑
"wąsy" (1.5×IQR)          outlier poza wąsem = kropka
```

> 🎯 Zasada wykrywania outlierów: wartość < Q1 − 1.5×IQR lub > Q3 + 1.5×IQR

🎲 Rozkłady prawdopodobieństwa — czym są?

Rozkład opisuje, jak często poszczególne wartości pojawiają się w danych.

Dlaczego to ważne? Bo znając rozkład, możesz:

- Przewidzieć, jak prawdopodobna jest dana wartość
- Wykryć, że coś jest "nietypowe" (outlier)
- Wybrać właściwy test statystyczny (za 2 tygodnie!)

```
import matplotlib.pyplot as plt

dane = np.random.normal(loc=250, scale=40, size=1200)
plt.hist(dane, bins=30, edgecolor="black")
plt.title("Histogram – kształt rozkładu")
plt.show()
```

🔔 Rozkład normalny (Gaussa)

Najsłynniejszy rozkład w statystyce — "krzywa dzwonowa".

```
czasy = np.random.normal(loc=250, scale=40, size=1200)  # μ=250 min, σ=40 min
```

Charakterystyka: symetryczny, większość wartości blisko średniej.

Reguła 68-95-99.7:

Zakres	% obserwacji
μ ± 1σ	~68%
μ ± 2σ	~95%
μ ± 3σ	~99.7%

> 🏃 Czasy ukończenia maratonu, wzrost ludzi, błędy pomiarowe — świetnie pasują do rozkładu normalnego!

🎯 Rozkład jednostajny (uniform)

Wszystkie wartości w zakresie są **równie prawdopodobne**.

```
numery_startowe = np.random.uniform(low=1, high=1500, size=1200)
```

```
Histogram:
|████████████████████████████████████████████████████████████|  ← płaski, bez "górki"
```

> 🎲 Klasyczny przykład: rzut uczciwą kostką — każda ściana ma taką samą szansę.

🎯 Rozkład Poissona

Liczba zdarzeń w ustalonym przedziale czasu/przestrzeni.

```
# Liczba zawodników mijających punkt kontrolny co minutę
zawodnicy_na_minute = np.random.poisson(lam=8, size=200)
```

> 🏃 Przykład: liczba biegaczy przekraczających metę w ciągu jednej minuty — wartości dyskretne, nieujemne.

Cechy: dyskretny, opisuje **rzadkie/losowe zdarzenia w czasie**, parametr λ = średnia liczba zdarzeń.

⏱️ Rozkład wykładniczy (exponential)

Czas oczekiwania **między** kolejnymi zdarzeniami.

```
# Czas (w sekundach) między kolejnymi zawodnikami mijającymi metę
odstepy_na_mecie = np.random.exponential(scale=7.5, size=1200)
```

> 🏁 Jeśli zawodnicy średnio mijają metę co 7.5 sekundy, rozkład wykładniczy opisze, jak zmienne są te odstępy — czasem 1s, czasem 40s!

Cechy: zawsze dodatni, silnie skośny w prawo, "brak pamięci" (memoryless).

🧭 EDA — co to właściwie jest?

EDA (Exploratory Data Analysis) = eksploracyjna analiza danych — pierwszy kontakt ze zbiorem danych.

Typowy workflow:

```
1. Wczytaj dane           → pd.read_csv() / generuj syntetycznie
2. Podejrzyj strukturę    → .head(), .info(), .dtypes
3. Sprawdź jakość danych  → braki, duplikaty, zakresy wartości
4. Policz statystyki      → .describe(), .mean(), .std()
5. Zwizualizuj            → histogram, boxplot, heatmapa
6. Wyciągnij wnioski      → co jest ciekawe? co trzeba naprawić?
```

> 💡 80% czasu w projekcie Data Science to właśnie EDA i czyszczenie danych. Nie żartuję. 😅

🔍 EDA krok 1 — pierwszy rzut oka

```
import pandas as pd

df = pd.DataFrame({
    "wiek": [24, 35, 41, 29, 52],
    "czas_min": [210, 245, 198, 320, 255],
})

print(df.shape)     # (5, 2) — wiersze, kolumny
df.info()            # typy danych, liczba non-null
df.head()             # pierwsze 5 wierszy
```

```
Output df.info():
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 2 columns):
 #   Column    Non-Null Count  Dtype
 0   wiek      5 non-null      int64
 1   czas_min  5 non-null      int64
```

🔍 EDA krok 2 — jakość danych

```
# Braki danych
print(df.isna().sum())

# Duplikaty
print(f"Liczba duplikatów: {df.duplicated().sum()}")

# Zakresy – czy ktoś "przebiegł" maraton w 5 minut?
print(df['czas_min'].min(), df['czas_min'].max())
```

> ⚠️ **Zawsze** sprawdzaj skrajne wartości — `min()` / `max()` ujawniają błędy w danych błyskawicznie!

🔍 EDA krok 3 — describe() to Twój najlepszy przyjaciel

```
print(df.describe())
```

```
             wiek    czas_min
count    5.000000    5.000000
mean    36.200000  245.600000
std     10.920166   45.869...
min     24.000000  198.000000
25%     29.000000  210.000000
50%     35.000000  245.000000
75%     41.000000  255.000000
max     52.000000  320.000000
```

> 🎯 Jedna linijka kodu = kompletna statystyka opisowa. Zapamiętaj `describe()` na zawsze!

📈 Wizualizacja — dlaczego "1000 słów"?

Te same dane, dwa różne rozkłady — **identyczne statystyki opisowe** (średnia, std)!

```
Zbiór A: symetryczny, jeden "garb"        Zbiór B: dwa "garby" (bimodalny)
```



> 😱 To zjawisko nazywa się **kwartet Anscombe'a** — dane mogą mieć te same liczby i wyglądać kompletnie inaczej. Zawsze rysuj wykres!

📊 Histogram w praktyce

```
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 6))
plt.hist(df['czas_min'], bins=30, edgecolor='black', alpha=0.7)
plt.axvline(df['czas_min'].mean(), color='red', linestyle='--',
            label=f"Średnia: {df['czas_min'].mean():.1f}")
plt.xlabel("Czas ukończenia (min)")
plt.ylabel("Liczba zawodników")
plt.title("Rozkład czasów ukończenia maratonu")
plt.legend()
plt.show()
```

> 🎯 Zawsze dodawaj tytuł, etykiety osi i (jeśli trzeba) legendę — wykres bez opisu to zagadka, nie analiza!

📦 Boxplot w praktyce — porównanie grup

```
import seaborn as sns

plt.figure(figsize=(10, 6))
sns.boxplot(data=df, x='kategoria_wiekowa', y='czas_min')
plt.title("Czas ukończenia wg kategorii wiekowej")
plt.xlabel("Kategoria wiekowa")
plt.ylabel("Czas ukończenia (min)")
plt.show()
```

> 💡 Boxplot obok siebie dla różnych grup = najszybszy sposób, by zobaczyć, kto biega szybciej: 20-latkowie czy 40-latkowie!

🔥 Heatmapa korelacji

```
import seaborn as sns

numeric_cols = ['wiek', 'czas_min', 'tempo_min_km']
corr = df[numeric_cols].corr()

sns.heatmap(corr, annot=True, cmap='coolwarm', center=0, vmin=-1, vmax=1)
plt.title("Macierz korelacji")
plt.show()
```

Wartość korelacji	Interpretacja
~ +1.0	Silna korelacja dodatnia
~ 0.0	Brak liniowej zależności
~ -1.0	Silna korelacja ujemna

> ⚠️ **Korelacja ≠ przyczynowość!** Więcej lodów sprzedanych ≠ więcej utonięć z powodu lodów. 🍦

🎨 Zasady dobrej wizualizacji

1. **Tytuł** — o czym jest ten wykres?
2. **Etykiety osi** — w jakich jednostkach?
3. **Legenda** — jeśli masz więcej niż jedną serię danych
4. **Odpowiedni typ wykresu** — histogram dla rozkładu, boxplot dla porównań, scatter dla zależności
5. **Nie przesadzaj z kolorami** — 3D pie chart to zbrodnia przeciw ludzkości 😤

```
plt.style.use('seaborn-v0_8-whitegrid')  # 👈 czysty, czytelny styl domyślny
```

⚠️ Pułapka #1 — średnia kłamie przy skośności

```
zarobki = np.array([3000, 3200, 3100, 2900, 500_000])  # jeden "wielki wygrany"
print(zarobki.mean())    # 102 640 — nikt tyle nie zarabia!
print(np.median(zarobki)) # 3100 — dużo bliżej prawdy
```

> 🎯 Zawsze pytaj: "czy w moich danych jest ktoś, kto ciągnie średnią w jedną stronę?"

⚠️ Pułapka #2 — korelacja to nie przyczynowość

Miasta z większą liczbą strażaków mają więcej pożarów.
Czy strażacy powodują pożary? Oczywiście nie — obie zmienne zależą od **wielkości miasta**! 🔥🚒

> 💡 To się nazywa **zmienna zakłócająca (confounding variable)**. Zawsze szukaj trzeciego czynnika, zanim ogłosisz odkrycie stulecia.

⚠️ Pułapka #3 — zbyt mała próba

```
mala_proba = np.random.normal(250, 40, size=5)   # tylko 5 obserwacji!
duza_proba = np.random.normal(250, 40, size=5000)

print(mala_proba.std())   # może wyjść cokolwiek – 15, 70, 5...
print(duza_proba.std())   # blisko 40 – stabilnie
```

> 🎲 Im mniejsza próba, tym większa **wariancja estymatora**. 5 zawodników nie powie Ci nic pewnego o 50 000 maratończyków na świecie.

🔬 Od opisu do wniosku — wnioskowanie statystyczne w pigułce

Statystyka opisowa mówi, co jest **w próbie**. Wnioskowanie pozwala powiedzieć coś o **populacji** — z kontrolowaną niepewnością.

Pojęcie	Intuicja
Przedział ufności (95% CI)	„Prawdziwa średnia leży gdzieś tutaj" — w 95% powtórzeń badania
H0 (hipoteza zerowa)	„Nie ma żadnej różnicy" — próbujemy ją obalić danymi
p-wartość	Jak bardzo zaskakujące są dane, **jeśli H0 jest prawdziwa**

> 🎯 Reguła kciuka: **p < 0.05** → odrzucamy H0, różnica raczej nie jest przypadkiem.

🔬 Test t w 3 linijkach

Czy młodsi biegacze są naprawdę szybsi, czy to przypadek?

```
from scipy import stats

mlodzi = df.loc[df["wiek"] < 35, "czas_min"]
starsi = df.loc[df["wiek"] >= 35, "czas_min"]
print(stats.ttest_ind(mlodzi, starsi, equal_var=False).pvalue)
```

> ⚠️ **Pułapka Big Data:** przy milionach rekordów niemal **każda** różnica wychodzi „istotna". Zawsze pytaj też: **czy ta różnica jest duża i ważna w praktyce?** 🤔

🏃 Real world tie-in — dlaczego to akurat maratony?

Organizatorzy dużych maratonów (Boston, Berlin, Nowy Jork) **codziennie** używają statystyki opisowej do:

- Planowania stref startowych wg tempa zawodników
- Wykrywania nietypowych czasów (podejrzenie o skróconą trasę!)
- Analizy wpływu pogody/wieku na wyniki
- Prognozowania czasu przybycia kolejnych fal biegaczy na metę

> 🎯 To dokładnie ten sam zestaw narzędzi, którego użyjesz dziś na ćwiczeniach — tylko dla prawdziwych zawodów!

📝 Podsumowanie — kluczowe wnioski

1. ✅ **Statystyka opisowa** = średnia, mediana, moda, wariancja, std, kwantyle
2. ✅ **Zawsze porównuj średnią i medianę** — duża różnica sygnalizuje skośność/outliery
3. ✅ **Rozkłady** (normalny, jednostajny, Poissona, wykładniczy) pomagają zrozumieć kształt danych
4. ✅ **EDA to workflow**: wczytaj → sprawdź jakość → policz → zwizualizuj → wyciągnij wnioski
5. ✅ **Wykres > statystyki** — zawsze rysuj, zanim wyciągniesz wnioski (kwartet Anscombe'a!)
6. ✅ **Wnioskowanie** — przedział ufności i p-wartość mówią, czy wynik z próby uogólnia się na populację

```
Twój workflow na dziś:
df.describe() → histogram → boxplot → heatmapa → wnioski 🎉
```

🔮 Co dalej?

Następny wykład (Wykład 2, 13.10.2026): Dane w skali — formaty i przechowywanie danych. Co się dzieje, gdy zamiast 1 200 maratończyków masz **miliardy rekordów**? CSV vs Parquet, partycjonowanie, Data Lake 📦

Do przeczytania:

- *Practical Statistics for Data Scientists* — Peter Bruce & Andrew Bruce (rozdz. 1-2)
- Dokumentacja pandas — `describe()`

Ćwiczenie:

- Ćwiczenie 1 — EDA na danych o maratonach amatorskich w Google Colab 🔲

❓ Pytania?

Dziękuję za uwagę! 🎉

Zanim pójdziecie na ćwiczenia — **jedno pytanie na rozgrzewkę:**

> Masz 1200 zawodników maratonu. Średni czas to 250 minut, odchylenie standardowe 40 minut.
> Ile procent zawodników ukończyło bieg między 210 a 290 minut? (Wskazówka: reguła 68-95-99.7!)

💬 Odpowiedzi na czacie lub na żywo!